A Comparative Report on the Loop Scheduling Options in OpenMP

Exam No. B024703

October 28, 2016

1 Introduction

This report details experiments undertaken to compare the performance of the loop scheduling options available in OpenMP. A serial code involving two separate loops was provided. This code measured and reported the time taken for each loop to execute 1000 times.

The nature of these loops led to different performance characteristics under parallelisation for each. This is due to the varying complexities of calculation within and between each of these loops. The variation in amounts of operations performed per loop cycle can cause issues with load balancing, where some threads complete all assigned work before others, leading to idling and sub-optimal performance.

The purpose of the loop scheduling options provided by OpenMP is to provide a range of strategies to improve load balancing between different threads. The remainder of this report will describe the methods used to measure the effectiveness of each of the loop schedules, before detailing the results of these experiments. Finally, this report will present a conclusion drawn from these results. Full results and optimised parallel code are included as appendices.

2 Methods

2.1 Compiling and Running Serial Code

As a precaution, to check the correctness of any subsequent parallel version, the original code was compiled in its serial form. This was done using the Intel C Compiler with the `-O3` option specified at compile time for maximum serial optimisation. Beyond using this to verify the correctness of any parallel solution and that parallel performance was credible, analysis of the performance of this serial code falls outside the scope of this report.

2.2 Parallelising the Code

Before different scheduling clauses could be used, the code had first to be parallelised. This was done using OpenMP directives and apart from these directives involved no alterations to the original source code. Within the directive for launching a parallel for loop, OpenMP gives the option of setting a schedule. The available schedule options are as follows:

- `static`
- `auto`
- `static,n`
- `dynamic,n`
- `guided,n`

where `n` specifies a chunksize.

The `static` loop schedule divides the loop into equal-sized chunks, or as evenly as possible where the number of loop iterations is not divisible by the number of threads multiplied by the chunksize. By default contiguous iterations assigned to each core. Setting a chunksize stripes the iterations in blocks of chunksize.

The `auto` schedule leaves scheduling options to the compiler. Intel does not provide documentation as to the implementation within their compliers.

The `dynamic,n` schedule uses an internal work queue to distribute blocks of chunksize to each core. When each core completes a block it is assigned another from the work queue. A similar technique is applied when the `guided` strategy is selected by the programmer, however in this case the block sizes start larger and then diminish in size. This theoretically reduces the overhead of work assignment, while allowing fine grain distribution towards the end of the computation, where load balancing issues are more likely to occur.

The Intel documentation [1] describes how the programmer has the option to set the schedule using an environment variable. This was done so that the schedule could be changed programmatically and that all experiments would be easily repeatable.

2.3 Compiling and Running the Code

As with the serial case the code was compiled using the Intel C Compiler and using the `-O3` optimisation flag. In this case, however, the appropriate compiler flag was used to compile an OpenMP program. To measure the performance of the different scheduling options the code was initially run on 4 threads on a backend node of `cirrus`, an Intel Xeon powered cluster. For the schedules where it is possible to set a chunksize, this chunksize took the values 1, 2, 4, 8, 32 and 64.

From these results, the scheduling option with the best performance for each loop on 4 threads was determined. Using these optimal options the code was then run again on 1, 2, 4, 6, 8, 12 and 16 threads. This allowed the measurement of the speed up, as defined by T_p/T_1, so that the scalability of the best performing schedule on 4 threads could be analysed.

For all measurements the running time was measured multiple times and a mean of these values used. In addition the standard deviation was also calculated. This allows the comparison of performance variability between different schedules. Performance variability is often an important consideration in HPC applications, where it may be desirable to avoid methods that often, but unreliably, yield high performance.

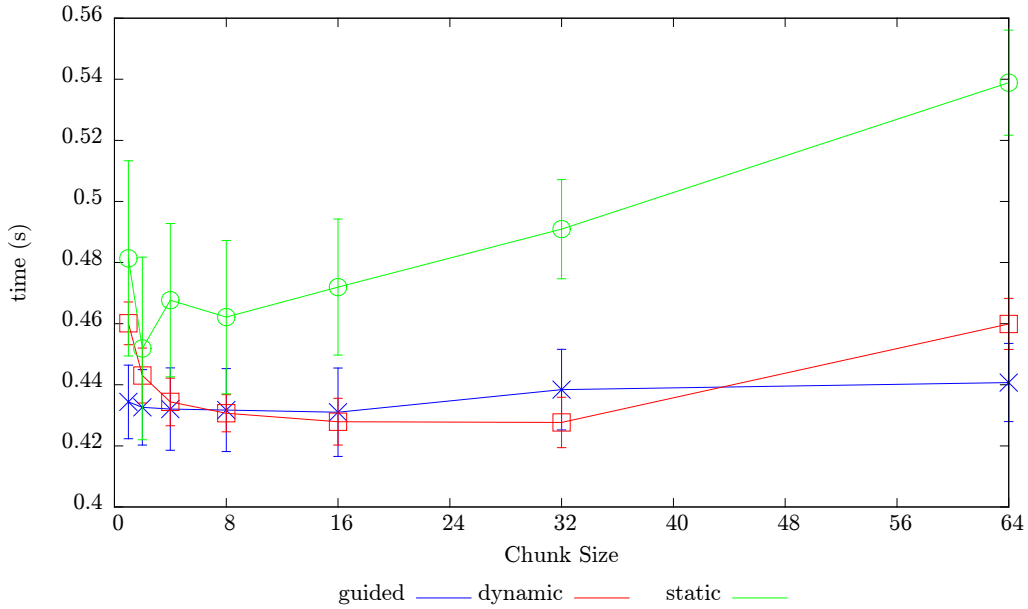

Figure 1: Graph showing execution time for loop 1 against block size for the guided, dynamic and static schedules. The error bars show 1 standard deviation. Note that these error bars are much larger than in loop 2.

3 Results

3.1 Correctness

For almost all HPC applications correctness is the primary concern. The serial code provided included a check sum that could be used to verify the correctness of future parallel results. In all parallel cases this sum yielded the same result as the serial code. We can therefore disregard correctness as a factor when comparing the different schedules.

3.2 Results of Schedules on 4 Threads

The first results gathered were execution times for each loop for every available loop schedule. The results for these are shown in Figures 1 and 2.

It is possible to use the results from the `static` schedule to give a good

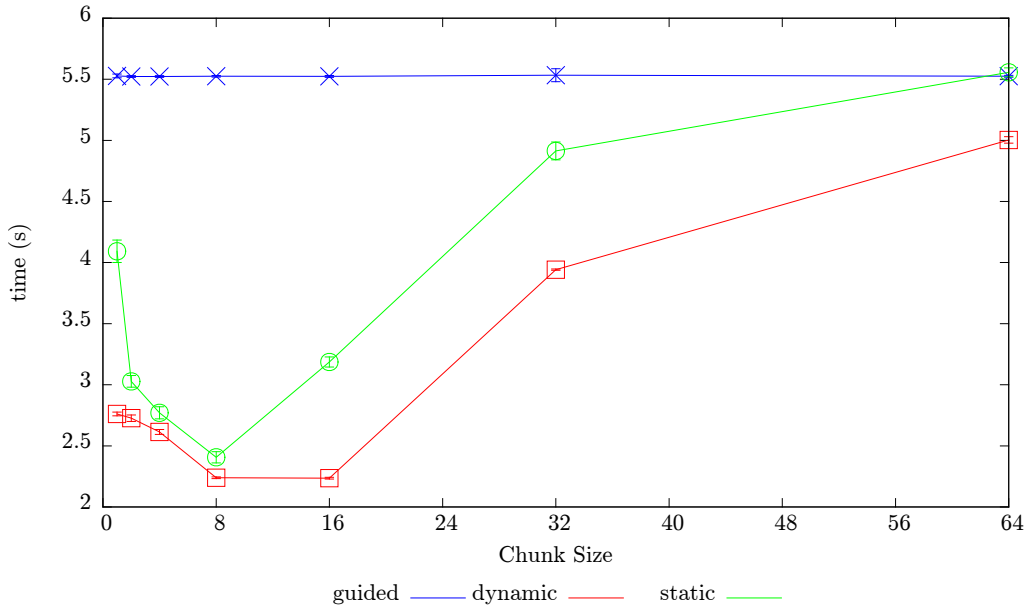

Figure 2: Graph showing execution time for loop 2 against chunk size for the guided, dynamic and static schedules.

experimental indication as to how evenly computational complexity is spread between iterations for the two loops. As the iterations are evenly distributed between threads with this `static` method we can compare its performance to the best schedule and see how poorly it performs compared with an optimally load balanced solution. Doing this we see that for loop 1 computational complexity is far better distributed between loop iterations than in loop 2. This ties in with static analysis of the code which shows that the number of floating point operations in each iteration of loop 2 is proportional to the index of the iteration.

For the loop 1 results there was a large standard deviation in all the schedules and chunk sizes. This shows a wide performance variability for this loop, particularly where the `static` schedule was used. For loop 2 much less variable results were achieved as shown by the small error bars showing standard deviation in Figure 2. This could be explained by the short excution time for 1000 repetitions of this loop is less than a second, at which point noise from I/O and other system processes can make results less reliable.

The highest performing schedule for loop 1 was found to be `dynamic,32`. This had the lowest mean execution time and also had one of the lowest

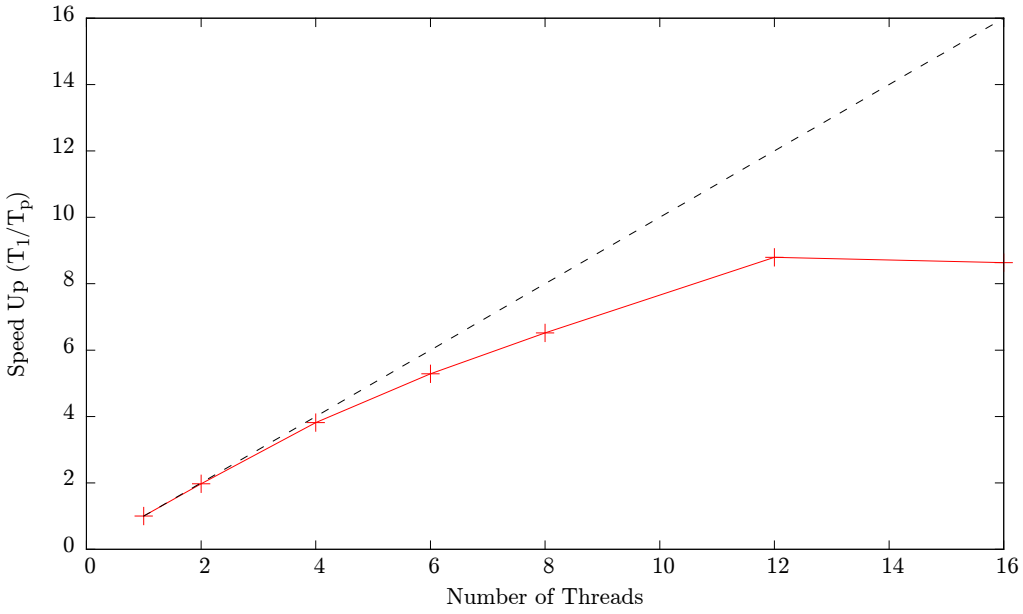

Figure 3: Speed Up of loop 1 for the dynamic 32 schedule. A theoretical ideal speed up is shown as a black dotted line.

standard deviations, meaning this schedule had reliably good performance for this loop.

For loop 2 we saw a general pattern where there was an optimal chunksize that was not so small that the overhead of work allocation became dominant, but not so large that the slowest chunk to complete was the dominant factor. This pattern was not followed by the `guided` schedule, which performed uniformly, and relatively badly, regardless of chunksize. This may be because this scheme starts by assigning large chunks from the work queue, meaning that a large slow chunk was the dominant factor for all chunksizes.

The highest performing schedule for loop 2 was not immediately obvious. The mean execution time with the `dynamic,8` and `dynamic,16` was similar to the order of nanoseconds - well beyond the accuracies of this experiment. It was therefore decided that both of these schedules would be jointly selected as the best.

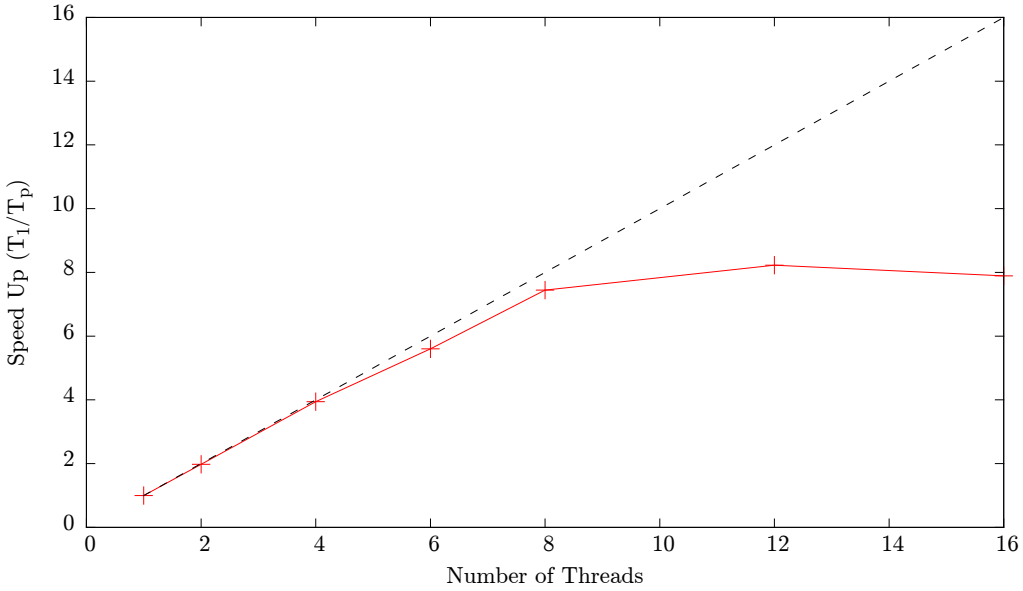

Figure 4: Speed Up of loop 2 for the dynamic 8 schedule. A theoretical ideal speed up is shown as a black dotted line.

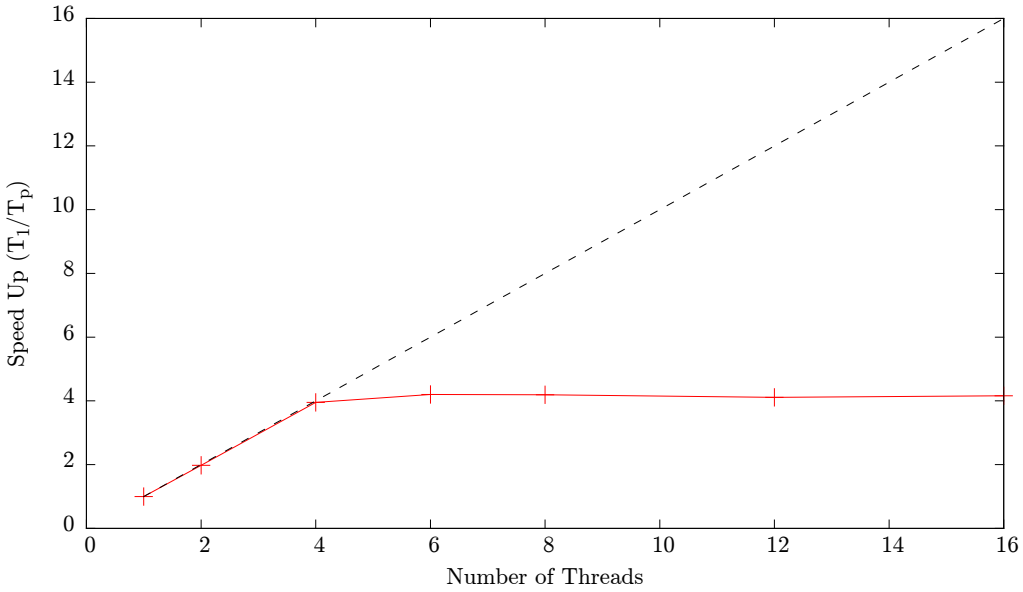

Figure 5: Speed Up of loop 2 for the dynamic 16 schedule. A theoretical ideal speed up is shown as a black dotted line.

3.3 Speed Up of Best Schedules

After the best schedules for loop 1 and loop 2 had been found they were then run again on 1, 2, 4, 6, 8, 12 and 16 threads. The speed up for loop 1 on the `dynamic,32` schedule is shown in Figure 3. Here relatively good scalability is shown until the system reaches 12 cores. At this point no more speed up is achieved. There are many reasons that could explain this, but as the chunksize is larger for this schedule. It could be that there are not enough blocks in the work queue to distribute between these threads, however considering the results for loop 2, detailed below, that the performance peaks at the time taken for the slowest chunk of 32 to execute.

The best schedule for loop 2 was taken to be either `dynamic,8` or `16`. Here Figures 4 and 5 show extremely interesting results. The scalability completely plateaus after 8 threads where the chunksize is 8, and 4 threads where the chunksize is 16. There are two plausible explanations for this. It could be that with the larger chunksizes there is a shortage of work in the work queue for these extra threads to do, as the total work is divided into fewer larger blocks. However, because this threshold is not reached by loop 1 where the loop is split into chunks of size 32, it seems more likely that that this is due to the time it takes for the slowest block to complete. As loop 2 is an inherently badly load balanced problem - the amount of flops per cycle is proportional to the loop index - it seems more likely that one of the threads is left with a larger block of these longer computations where the chunksize is bigger, and however well distributed the remainder of the problem is the loop cannot complete before this longest problem is solved. What we see, then, is the size of this most computationally dense chunk doubling as the chunksize doubles, leaving no further room for improvement however many threads are used.

Comparing these two loops we can see that in a more evenly distributed loop a larger chunksize is better. This is explained by the overhead from each thread going to the work queue. This becomes the dominant factor where the time taken for each chunk to complete is approximately even.

4 conclusion

To conclude, the results found in this experiment shows that loop schedules deserve considerable attention from a programmer trying to achieve high performance with an OpenMP code. A poor selection was shown to increase execution time in some cases by more than a factor of 2.

The results also demonstrate there are a balance of factors to consider when selecting the the optimal schedule. The experiments showed that the assignment of work from a queue has an overhead. It was also shown that performance could be limited by poor load balancing where a chunksize that was too large was selected. This had a severe impact on horizontal scaling.

Specifically in this case, results showed that the optimal scheduling strategy was for `dynamic,32` for loop 1 and `dynamic,8` for loop 2.

More generally it was shown that the optimal schedule will depend on the number of threads that the program will be run on as well as the nature of the loop being parallelised. It would therefore be advisable for any HPC programmer to conduct some experiments for there specific use cases before running a large job.

References

[1] Intel Corp. *OpenMP Loop Scheduling* `https://software.intel.com/en-us/articles/openmp-loop-scheduling` Accessed: 26-10-2016

A Full Results

Table 1: Loop 1

Schedule	No. Threads	Mean Time (s)	Std. Deviation (s)
static	4	0.728231	0.00666976
auto	4	0.430124	0.00860091
static,1	4	0.481388	0.0319373
static,2	4	0.451891	0.0298839
static,4	4	0.467681	0.0250977
static,8	4	0.462157	0.0250929
static,16	4	0.471986	0.0222526
static,32	4	0.490942	0.0162345
static,64	4	0.538883	0.0171833
dynamic,1	4	0.460126	0.0069964
dynamic,2	4	0.443026	0.00900873
dynamic,4	4	0.434388	0.00780827
dynamic,8	4	0.430674	0.00608488
dynamic,16	4	0.42792	0.00766377
dynamic,32	4	0.427667	0.0082635
dynamic,64	4	0.459919	0.00837936
guided,1	4	0.434381	0.0120444
guided,2	4	0.432622	0.0123446
guided,4	4	0.432052	0.0134753
guided,8	4	0.431721	0.0135889
guided,16	4	0.43101	0.0144585
guided,32	4	0.438409	0.0131864
guided,64	4	0.440735	0.0127809
dynamic,8	1	1.64584	0.00368912
dynamic,8	2	0.842835	0.000531856
dynamic,8	4	0.432655	0.0101219
dynamic,8	6	0.300801	0.0106516
dynamic,8	8	0.229031	0.00784543
dynamic,8	12	0.159318	0.00782003
dynamic,8	16	0.126953	0.00697052
dynamic,16	1	1.6453	0.000578072
dynamic,16	2	0.837223	0.00103113
dynamic,16	4	0.432996	0.0128874
dynamic,16	6	0.303278	0.0114546
dynamic,16	8	0.23483	0.00944744
dynamic,16	12	0.162625	0.00981424
dynamic,16	16	0.138232	0.00830332

Table 2: Loop 2

Schedule	No. Threads	Mean Time (s)	Std. Deviation (s)
static	4	6.35165	0.0350881
auto	4	5.5247	0.0176731
static,1	4	4.09267	0.0922924
static,2	4	3.02777	0.0478233
static,4	4	2.7703	0.0496694
static,8	4	2.4068	0.0457846
static,16	4	3.18633	0.0415496
static,32	4	4.91394	0.0732637
static,64	4	5.55721	0.0367112
dynamic,1	4	2.76134	0.0153146
dynamic,2	4	2.72649	0.0267236
dynamic,4	4	2.61394	0.0205074
dynamic,8	4	2.23896	0.007317
dynamic,16	4	2.23559	0.00661029
dynamic,32	4	3.94226	0.00563607
dynamic,64	4	5.00349	0.0262898
guided,1	4	5.52794	0.0151542
guided,2	4	5.52264	0.00682783
guided,4	4	5.52289	0.00714884
guided,8	4	5.52478	0.00667864
guided,16	4	5.52349	0.00793705
guided,32	4	5.5337	0.0532817
guided,64	4	5.52464	0.00727738
dynamic,32	1	8.84485	0.0103624
dynamic,32	2	4.46239	0.00300086
dynamic,32	4	3.94598	0.00589791
dynamic,32	6	3.94177	0.00235488
dynamic,32	8	3.9493	0.00520081
dynamic,32	12	3.98098	0.0194448
dynamic,32	16	3.95387	0.00761888